

13 - Comprensión, Zip y Tateti

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Río Negro



Comprensión de listas, conjuntos y diccionarios

Python nos ofrece más de un mecanismo para recorrer estos elementos y transformarlos.

Listas por comprensión

La comprensión de listas es una forma compacta de construir una lista a partir de un iterable

Con for clásico:

```
cuadrados = []  
for x in range(5):  
    cuadrados.append(x ** 2)
```

Con comprensión:

```
cuadrados = [x ** 2 for x in range(5)]
```

La estructura general es:

```
[expresion for variable in iterable]
```

Diccionarios por comprensión

La comprensión de diccionarios es una forma compacta de construir una diccionarios a partir de un iterable

La estructura general es igual a las listas pero con la diferencia de que se utilizan las llaves "{}" tan representativas de los diccionarios:

```
# {key: value for variable in iterable}
```

```
# Con for clasico
cuadrados = {}
for x in range(5):
    cuadrados[x] = x ** 2
print(cuadrados)

# Con comprensión:
{x: x ** 2 for x in range(5)}
```

Conjuntos por comprensión

La comprensión de conjuntos es una forma compacta de construir una diccionarios a partir de un iterable

Estructura similar al diccionario pero no se debe indicar clave valor para poder generar el conjunto:

```
# {expresion for variable in iterable}
```

```
# Con for clasico
cuadrados = {}
for x in range(5):
    cuadrados.add(x ** 2)
print(cuadrados)
```

```
# Con comprensión:
{x ** 2 for x in range(5)}
```

comprensiones anidadas

Se pueden tener varios ciclos for dentro de una misma comprensión.

```
# Con for clásico:
```

```
pares = []  
for x in [1, 2, 3]:  
    for y in ['a', 'b']:  
        pares.append((x, y))
```

```
# Con comprensión:
```

```
pares = [(x, y) for x in [1, 2, 3] for y in ['a', 'b']]
```

```
trios = [(x, y, z) for x in [1, 2, 3] for y in ['a', 'b'] for z in [3, 2, 1]]
```

Condicionando el iterador de estructuras de comprensión

Podemos también utilizar una condición la comprensión de elementos.

Esto aplica para todos los mecanismos de comprensión que vimos anteriormente.

Este mecanismo nos permite condicionar la lista iterada.

Usamos `if` al final si querés que la lista o diccionario final sea más chico que el original (Filtrar)

```
# Obtener lista de cuadrados pares con for clasico
```

```
cuadrados = []
```

```
for x in range(5):
```

```
    if (x ** 2) % 2 == 0:
```

```
        cuadrados.append(x ** 2)
```

```
# Obtener lista de cuadrados pares con comprensión
```

```
cuadrados = [x ** 2 for x in range(5) if (x ** 2) % 2 == 0]
```

```
# Obtener diccionario de cuadrados pares con for clasico
```

```
cuadrados = {}
```

```
for x in range(5):
```

```
    if (x ** 2) % 2 == 0:
```

```
        cuadrados[x] = x ** 2
```

```
# Obtener diccionario de cuadrados pares con comprensión
```

```
{x: x ** 2 for x in range(5) if (x ** 2) % 2 == 0}
```

Condicionando expresión en estructuras de comprensión

Podemos también utilizar una condición durante la generación de la salida.

Esto aplica para todos los mecanismos de comprensión que vimos anteriormente.

Este mecanismo nos permite condicionar los elementos de la lista iterada.

Usamos `if ... else` al principio si querés que la lista o diccionario final tenga el mismo tamaño, pero con datos transformados (Modificar).

```
notas = [4, 7, 5, 9, 6]
```

```
# Con comprensión
```

```
estados = ["Aprobado" if n >= 6 else "Desaprobado" for n in notas]
```

```
# Con for
```

```
estados = []
```

```
for nota in notas:
```

```
    if nota >= 6:
```

```
        estados.append("Aprobado")
```

```
    else:
```

```
        estados.append("Desaprobado")
```

```
productos = {"Teclado": 120, "Mouse": 80, "Monitor": 450}
```

```
# Con comprensión
```

```
nuevos_precios = {prod: (precio * 0.9 if precio > 100 else precio) for prod,  
precio in productos.items() }
```

```
# Con for
```

```
productos_precios_nuevos = {}
```

```
for producto, precio in productos.items():
```

```
    if precio > 100:
```

```
        productos_precios_nuevos[producto] = precio * 0.9
```

```
    else:
```

```
        productos_precios_nuevos[producto] = precio
```

Advertencia

Si bien las comprensiones en Python nos permiten reducir la cantidad de código, no siempre son la mejor opción.

Su uso excesivo puede afectar la legibilidad y hacer que los programas sean más difíciles de mantener.

Por eso, conviene usarlas con criterio: el código debería entenderse a simple vista. Si después de escribir una comprensión la leemos y no queda claro cómo funciona, probablemente sea mejor usar una versión más tradicional.

Algunos ejercicios de comprensión

1. Mediante comprensión normaliza los textos de la siguiente lista:

```
comandos = [" ENCENDER ", "apagar", " Estado ",  
"REINICIAR", " salir "]
```

2. Mediante comprensión devuelve una lista que devuelva True para los aprobados (mayor o igual a 6) o False para los desaprobados.

```
notas = [2, 4, 6, 8, 10, 3, 7, 9]
```

3. Transformar en una lista mediante comprensions

```
mediciones = [3.2, 2.8, 4.1, 5.5, 3.0, 6.2, 4.8]
```

```
fuera_de_rango = []
```

```
for valor in mediciones:  
    if valor < 3.0 or valor > 5.0:  
        fuera_de_rango.append(valor)
```

```
print(fuera_de_rango)
```

ZIP

ZIP

Uniendo elementos iterables

La función zip nos permite obtener tuplas que combinen el resultado de múltiples variables iterables.

```
list(zip([1, 2], ["Hola", "Mundo"]))  
# [(1, 'Hola'), (2, 'Mundo')]  
  
# Se puede utilizar con más de dos elementos  
list(zip(  
    [1, 2],  
    ["Hola", "Adios"],  
    ["Buenas", "Nos vimos"]  
))  
# [(1, 'Hola', 'Buenas'), (2, 'Adios', 'Nos vimos')]  
  
# El iterador más corto va a limitar el zip  
list(zip(  
    [1, 2],  
    ["Hola", "Adios"],  
    ["Buenas"]  
))  
# [(1, 'Hola', 'Buenas')]
```

¿Para qué usamos un ZIP?

```
# RECORRER DOS LISTAS EN PARALELO
```

```
nombres = ["Juana", "Luis", "Marta"]
```

```
edades = [28, 34, 25]
```

```
for nombre, edad in zip(nombres, edades):  
    print(f"{nombre} tiene {edad} años")
```

```
# CONSTRUIR DICCIONARIOS
```

```
claves = ["nombre", "apellido", "edad"]
```

```
valores = ["Juana", "Martinez", 29]
```

```
config = dict(zip(claves, valores))
```

```
print(config)
```

```
# TRANSPONER UNA MATRIZ
```

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

```
transpuesta = list(zip(*matriz))
```

```
print(transpuesta)
```

Ejercicios ZIP

Imprimi los cambios en el STOCK entre ayer y hoy utilizando ZIP.

Tenemos las siguientes listas:

```
productos = ["mouse", "teclado", "monitor",  
"notebook"]  
stock_ayer = [10, 5, 3, 2]  
stock_hoy = [7, 8, 3, 1]
```

Construir una lista de diccionarios llamada usuarios utilizando ZIP

```
keys = ["id", "email", "age", "active"]  
  
values = [  
    [1, "ana@mail.com", 28, True],  
    [2, "luis@mail.com", 35, True],  
    [3, "marcos@mail.com", 31, False],  
    [4, "marta@mail.com", 25, True],  
]
```

Creando un tateti

PDFS y TAREAS



Todo el material en <https://github.com/Ymil/unrn-programacion-1/>

FIN

Volvemos el miércoles a las 17